宠物用品模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 商品表(product)

字段名	数据类型	描述	约束条件
id	BIGINT	商品ID	主键，自增
name	VARCHAR(100)	商品名称	非空
category	VARCHAR(50)	商品分类	非空
price	DECIMAL(10,2)	商品价格	非空
stock	INT	商品库存	非空
description	TEXT	商品描述	可选
images	VARCHAR(1000)	商品图片URL，多个图片用逗号分隔	可选
status	INT	商品状态：0下架，1上架	非空，默认1
create_time	DATETIME	创建时间	自动生成
update_time	DATETIME	更新时间	自动更新

Product类：对应商品表，包含商品基本信息
【springboot/src/main/java/org/example/springboot/entity/Product.java】

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@TableName("product")
@Schema(description = "商品信息")
public class Product {
```

半夜撕代码版权所有

```
    @TableId(type = IdType.AUTO)
    @Schema(description = "商品ID")
    private Long id;

    @Schema(description = "商品名称")
    private String name;

    @Schema(description = "商品分类")
    private String category;

    @Schema(description = "商品价格")
    private BigDecimal price;

    @Schema(description = "商品库存")
    private Integer stock;

    // 其他属性...
}
```

1.2 核心属性说明

1. **商品分类**：包括主粮、零食、玩具、日用品、保健品、服饰等多个分类，方便用户按需筛选
2. **商品图片**：支持多图存储，images字段中存储图片URL，多个图片之间使用逗号分隔
3. **商品状态**：通过status字段（1:上架，0:下架）控制商品是否可被用户浏览和购买
4. **库存管理**：stock字段实时反映商品剩余库存，避免超卖情况

二、各功能详细讲解

2.1 商品列表浏览功能

2.1.1 功能概述

商品列表浏览功能允许用户查看所有上架的宠物用品，支持按名称和分类筛选，并提供分页功能。界面采用卡片式布局，展示商品图片、名称、价格和库存状态，方便用户快速浏览和选择。

2.1.2 详细讲解功能实现流程

用户访问商品列表页面时，前端通过ProductList.vue组件初始化，在mounted钩子中调用fetchProducts方法。该方法通过request.get('/product/page')发送请求，传递搜索条件、分页参数和状态筛选条件。

后端ProductController接收请求，调用productService.getProductsByPage方法处理分页查询。ProductService构建查询条件，如商品名称模糊查询、分类精确匹配和状态筛选，然后通过MyBatis-Plus的分页功能执行查询。

半夜撕代码版权所有

查询结果返回前端后，ProductList.vue将数据渲染为商品卡片网格。每张卡片显示商品第一张图片、分类、名称和价格，还会根据库存状态显示不同的库存标签。用户可以点击商品卡片查看详情，或直接添加到购物车。

2.1.3 关键代码讲解

1. 前端商品列表展示
   【vue3/src/views/frontend/product/ProductList.vue】

```
// 获取商品列表
const fetchProducts = async () => {
  loading.value = true
  try {
    await request.get('/product/page', {
      name: searchForm.name,
      category: searchForm.category,
      currentPage: currentPage.value,
      size: pageSize.value,
      status: 1
    }, {
      onSuccess: (res) => {
        productList.value = res.records
        total.value = res.total
      }
    })
  } catch (error) {
    console.error('获取商品列表失败:', error)
  } finally {
    loading.value = false
  }
}
```

这段代码实现了商品列表的获取逻辑。通过request工具发送GET请求，传递搜索条件和分页参数，成功后将返回的记录和总数更新到响应式变量中，驱动界面渲染。

2. 后端分页查询实现
   【springboot/src/main/java/org/example/springboot/service/ProductService.java】

```
public Page<Product> getProductsByPage(String name, String category,
Integer status, Integer currentPage, Integer size) {
    LambdaQueryWrapper<Product> queryWrapper = new LambdaQueryWrapper<>
();

    // 添加查询条件
    if (StringUtils.hasText(name)) {
        queryWrapper.like(Product::getName, name);
    }
    if (StringUtils.hasText(category)) {
```

半夜撕代码版权所有

```
            queryWrapper.eq(Product::getCategory, category);
        }

        // 默认只查询上架商品
        if(status != null) {
            queryWrapper.eq(Product::getStatus, status);
        }

        // 按ID降序排列
        queryWrapper.orderByDesc(Product::getId);

        return productMapper.selectPage(new Page<>(currentPage, size),
queryWrapper);
    }
```

这段代码实现了商品分页查询的核心逻辑。根据前端传入的名称和分类构建查询条件，支持模糊查询和精确匹配，结合状态筛选确保只返回上架商品，最后通过MyBatis-Plus的分页功能执行分页查询。

2.2 商品详情查看功能

2.2.1 功能概述

商品详情功能提供商品的全面信息展示，包括多图轮播、价格、库存、商品描述和用户评价等。用户可以在此页面选择商品数量，添加到购物车或直接购买，还可以查看其他用户的评价参考。

2.2.2 详细讲解功能实现流程

用户点击商品卡片后，前端路由跳转到ProductDetail.vue组件，并传递商品ID。组件在created钩子中调用fetchProductDetail方法，通过request.get(`/product/${id}`)获取商品详情。

后端ProductController接收请求，调用productService.getProductById方法获取商品信息。方法首先检查商品是否存在，不存在则抛出异常，存在则返回商品详情。

前端接收到数据后，将商品信息渲染到页面，包括商品图片轮播、价格、库存、描述等。同时，页面还会加载商品评价信息。用户可以选择商品数量，点击"添加到购物车"或"立即购买"按钮，触发相应的购物流程。

2.2.3 关键代码讲解

1. **前端商品详情获取**
   【vue3/src/views/frontend/product/ProductDetail.vue】

```
const fetchProductDetail = async () => {
  loading.value = true
  try {
    await request.get(`/product/${route.params.id}`, {}, {
```

```
        onSuccess: (data) => {
          product.value = data
          quantity.value = 1

          // 处理商品图片
          if (data.images) {
            productImages.value = data.images.split(',')
          } else {
            productImages.value = []
          }

          // 生成图片URL列表
          productImageUrls.value = productImages.value.map(img =>
getImageUrl(img))
        }
      })
    } catch (error) {
      console.error('获取商品详情失败:', error)
    } finally {
      loading.value = false
    }
  }
```

这段代码实现了商品详情的获取和处理。通过路由参数获取商品ID，调用后端接口获取商品信息，然后处理商品图片字符串，将其分割为数组便于轮播展示。

2. **后端商品详情获取**
   【springboot/src/main/java/org/example/springboot/service/ProductService.java】

```
public Product getProductById(Long id) {
    Product product = productMapper.selectById(id);
    if (product == null) {
        throw new ServiceException("商品不存在");
    }
    return product;
}
```

这段代码实现了根据ID获取商品详情的逻辑。通过MyBatis-Plus的selectById方法查询商品，如果不存在则抛出异常，存在则返回商品对象。

2.3 购物车管理功能

2.3.1 功能概述

购物车功能允许用户临时存储想要购买的商品，支持添加、删除、修改数量和清空购物车等操作。用户可以在购物车中查看所选商品的总价和总数量，然后一键结算生成订单。

半夜撕代码版权所有

2.3.2 详细讲解功能实现流程

用户点击商品卡片或详情页的"加入购物车"按钮时，前端调用addToCart方法，该方法验证商品库存后将商品信息添加到购物车。购物车数据存储在前端localStorage中，保证页面刷新后数据不丢失。

用户进入购物车页面时，Cart.vue组件从localStorage加载购物车数据，并渲染为商品表格。用户可以修改商品数量，系统会自动重新计算商品小计和总价。用户也可以删除商品或清空购物车。

当用户点击"结算"按钮时，系统打开结算对话框，用户填写收货地址、联系人、联系电话等信息，选择支付方式后提交订单。前端将购物车数据和收货信息一并发送到后端，创建新订单并清空购物车。

2.3.3 关键代码讲解

1. 添加商品到购物车
   【vue3/src/views/frontend/product/ProductDetail.vue】

```
const addToCart = async () => {
  if (!userStore.isLoggedIn) {
    ElMessage.warning('请先登录')
    router.push('/login')
    return
  }

  if (product.value.stock <= 0) {
    ElMessage.warning('商品库存不足')
    return
  }

  addingToCart.value = true
  try {
    // 从本地存储获取购物车
    const cartData = JSON.parse(localStorage.getItem('cart') ||
'{"items": [], "totalAmount": 0, "totalQuantity": 0}')

    // 检查商品是否已在购物车中
    const existingItem = cartData.items.find(item => item.productId ===
product.value.id)

    if (existingItem) {
      // 更新数量
      existingItem.quantity += quantity.value
      existingItem.totalAmount = existingItem.price *
existingItem.quantity
    } else {
      // 添加新商品
      cartData.items.push({
```

```
        productId: product.value.id,
        productName: product.value.name,
        productImage: product.value.images ?
product.value.images.split(',')[0] : '',
        price: product.value.price,
        quantity: quantity.value,
        totalAmount: product.value.price * quantity.value
      })
    }

    // 重新计算总价和总数量
    cartData.totalAmount = cartData.items.reduce((sum, item) => sum +
item.totalAmount, 0)
    cartData.totalQuantity = cartData.items.reduce((sum, item) => sum +
item.quantity, 0)

    // 保存到本地存储
    localStorage.setItem('cart', JSON.stringify(cartData))

    ElMessage.success('已添加到购物车')
  } catch (error) {
    console.error('添加到购物车失败:', error)
    ElMessage.error('添加到购物车失败')
  } finally {
    addingToCart.value = false
  }
}
```

这段代码实现了添加商品到购物车的逻辑。首先检查用户登录状态和商品库存，然后从localStorage获取购物车数据，检查商品是否已存在，存在则更新数量，不存在则添加新商品，最后重新计算总价和总数量，并保存回localStorage。

2.4 商品管理功能

2.4.1 功能概述

商品管理功能是后台管理系统的核心功能之一，允许管理员对商品进行全面管理，包括添加、编辑、上下架和删除商品。系统提供搜索和筛选功能，方便管理员快速定位商品。

2.4.2 详细讲解功能实现流程

管理员登录后台系统后，通过ProductManage.vue组件进行商品管理。组件初始化时调用fetchProducts方法获取商品列表，支持按名称、分类和状态筛选。

添加商品时，管理员点击"添加商品"按钮，打开对话框填写商品信息，包括名称、分类、价格、库存、状态和上传图片等。提交后，前端将数据发送到后端ProductController的createProduct接口。

半夜撕代码版权所有

编辑商品时，系统先加载商品现有信息到表单，管理员修改后提交，前端调用updateProduct接口更新商品。上下架商品时，系统直接修改商品状态并保存。删除商品时，系统弹出确认对话框，确认后调用deleteProduct接口删除商品。

2.4.3 关键代码讲解

1. **后端商品创建逻辑**
   【springboot/src/main/java/org/example/springboot/service/ProductService.java】

```java
@Transactional
public boolean createProduct(Product product) {
    // 设置默认值
    if (product.getStatus() == null) {
        product.setStatus(1); // 默认上架
    }
    product.setCreateTime(LocalDateTime.now());
    product.setUpdateTime(LocalDateTime.now());

    return productMapper.insert(product) > 0;
}
```

这段代码实现了商品创建的核心逻辑。设置商品默认状态为上架，自动填充创建时间和更新时间，然后通过MyBatis-Plus的insert方法插入数据库。

2. **前端商品管理表格**
   【vue3/src/views/backend/product/ProductManage.vue】

```vue
<el-table
  v-loading="loading"
  :data="productList"
  border
  style="width: 100%"
>
  <el-table-column prop="id" label="ID" width="80"></el-table-column>
  <el-table-column label="商品图片" width="100">
    <template #default="scope">
      <el-image
        :src="getImageUrl(scope.row.images)"
        style="width: 60px; height: 60px"
        :preview-src-list="[getImageUrl(scope.row.images)]"
        fit="cover"
        :preview-teleported="true"
      >
      </el-image>
    </template>
  </el-table-column>
  <!-- 其他列... -->
  <el-table-column label="操作" width="200" fixed="right">
```

半夜撕代码版权所有

```
    <template #default="scope">
      <el-button type="primary" size="small"
@click="handleEdit(scope.row)">编辑</el-button>
      <el-button
        :type="scope.row.status === 1 ? 'warning' : 'success'"
        size="small"
        @click="handleChangeStatus(scope.row)"
      >
        {{ scope.row.status === 1 ? "下架" : "上架" }}
      </el-button>
      <el-button type="danger" size="small"
@click="handleDelete(scope.row)">删除</el-button>
    </template>
  </el-table-column>
</el-table>
```

这段代码展示了商品管理表格的实现。表格展示商品基本信息，包括ID、图片、名称、分类、价格、库存和状态等，并提供编辑、上下架和删除操作按钮。

三、总结

3.1 商品列表浏览功能实现

商品列表浏览功能通过前端ProductList.vue组件和后端ProductController实现。用户访问商品列表页面时，前端发送请求获取商品数据，支持按名称和分类筛选，并提供分页功能。后端通过构建查询条件，实现了灵活的商品筛选和分页查询。界面采用卡片式布局，直观展示商品信息，并根据库存状态显示不同的标签，提升用户体验。

3.2 商品详情查看功能实现

商品详情功能通过ProductDetail.vue组件展示商品的详细信息。当用户点击商品卡片时，系统通过路由参数获取商品ID，调用后端接口获取商品详情。页面使用轮播图展示商品多张图片，清晰展示价格、库存和描述信息，并提供数量选择和加入购物车功能。商品详情页还集成了用户评价系统，方便用户参考他人购买体验。

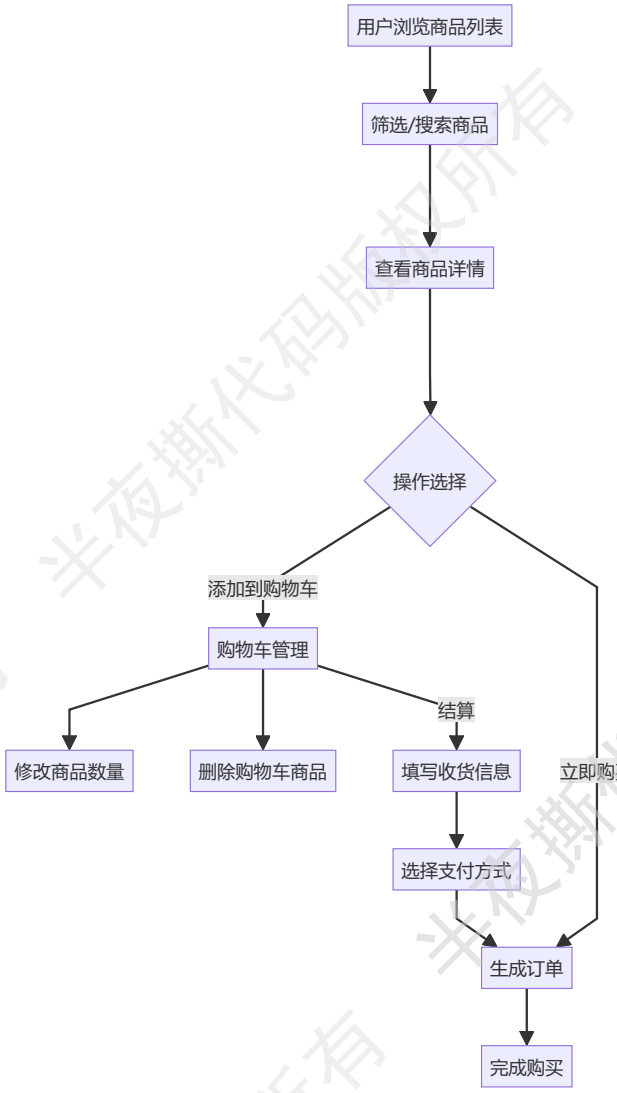
3.3 购物车管理功能实现

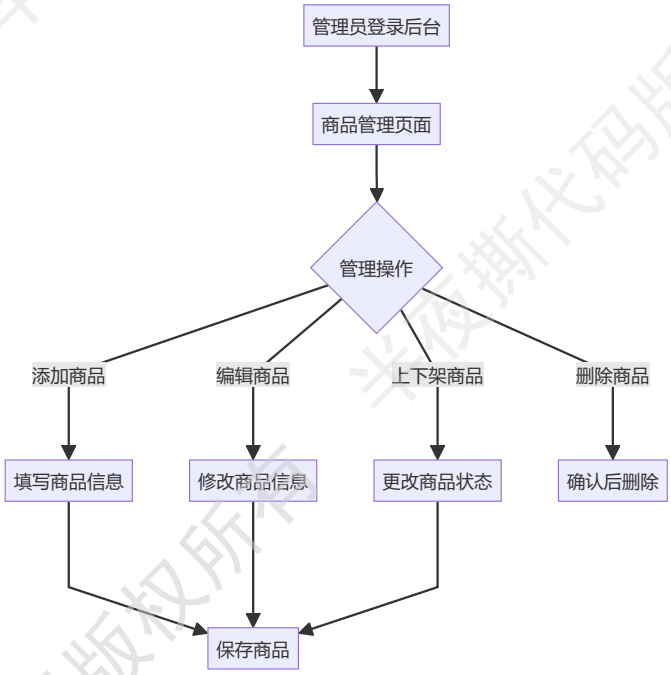
购物车功能采用前端localStorage存储方案，实现了添加商品、修改数量、删除商品和清空购物车等操作。Cart.vue组件负责展示购物车内容，自动计算每件商品的小计和购物车总价。结算流程整合了地址管理和支付方式选择，提供了完整的购物体验。购物车数据在用户下单后转化为订单，实现了从商品浏览到下单的完整流程。

半夜撕代码版权所有

3.4 商品管理功能实现

商品管理功能通过ProductManage.vue组件为管理员提供了全面的商品管理能力。管理员可以添加新商品，编辑现有商品信息，上下架商品或删除商品。界面提供了高效的搜索和筛选工具，方便管理员快速定位商品。商品图片上传功能简化了图片管理流程，支持多图上传和预览，提升了管理效率。





半夜撕代码版权所有